ASSIGNMENT- 04

Bachelor of Computer Applications (Hons. AI & ML)
Semester- III

Subject: Database Management System

Topic: Relational Database vs. NoSQL Database

By
Name: MERIN THOMAS
Reg. No.: 2511021240062

Faculty Name: RENITA JOY
Faculty Signature:_____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
August 2026

1. INTRODUCTION

Databases are essential for storing, organizing, retrieving, and managing information in modern applications. Two widely used database approaches are Relational Database Management Systems (RDBMS) and NoSQL databases.

RDBMS organizes data into tables containing rows and columns, while NoSQL databases support different data models, including document, key-value, column-family, and graph databases.

Both approaches have advantages and limitations. The choice depends on the application's requirements, data structure, consistency needs, scalability, and performance.

This assignment compares RDBMS and NoSQL databases and explains their applications, differences, and suitability for the Paranormal Investigation Agency Management System.

2. WHAT IS AN RDBMS?

A Relational Database Management System (RDBMS) stores data in related tables. Each table contains rows and columns, and relationships are maintained using primary keys and foreign keys.

RDBMS commonly uses Structured Query Language (SQL) to create tables, insert records, retrieve information, and update or delete data.

Examples of RDBMS

- **MySQL**
- **PostgreSQL**
- **Oracle Database**
- **Microsoft SQL Server**

Features of RDBMS

1. Structured data: Information is organized into tables with predefined columns.
2. Relationships: Primary keys and foreign keys connect related tables.
3. Data integrity: Constraints help prevent invalid or inconsistent records.
4. Transactions: Transactions can follow ACID properties to maintain reliable data operations.
5. SQL support: SQL allows users to retrieve and manipulate data efficiently.
6. Normalization: Tables can be organized to reduce unnecessary data duplication.

Example

In the Paranormal Investigation Agency Management System, client information can be stored in a Clients table.

client_id	name	phone
101	Anita Roy	9876501001
102	Rohan Das	9876501002
103	Meera Joseph	9876501003

Each client has a unique client_id. The Cases table can use this identifier as a foreign key to connect cases with their respective clients.

This structure makes it easier to maintain client information without repeating it unnecessarily.

3. WHAT IS A NOSQL DATABASE?

A NoSQL database is a non-relational database designed to store and manage data using models other than the traditional relational table structure.

NoSQL databases may support flexible schemas, large-scale distributed storage, and different methods of organizing information.

Types of NoSQL Databases

1. Document Database

Stores information as documents, commonly in JSON-like formats.

Example: MongoDB.

Use case: Storing investigation reports that contain different types of details or evidence descriptions.

2. Key-Value Database

Stores data as key-value pairs, where each key identifies a value.

Example: Redis.

Use case: Storing temporary session information or frequently accessed application settings.

3. Column-Family Database

Organizes data into column families and is designed to handle large amounts of distributed data.

Example: Apache Cassandra.

Use case: Managing large volumes of activity records collected from multiple investigation devices.

4. Graph Database

Stores information as nodes and relationships between nodes.

Example: Neo4j.

Use case: Exploring relationships between locations, investigators, cases, and recurring evidence patterns.

Features of NoSQL

1. Flexible data models: Different records can contain different attributes, depending on the database model.
2. Horizontal scalability: Many NoSQL systems can scale by distributing data across multiple machines.
3. High-volume data handling: Some NoSQL databases are designed for large amounts of distributed data.
4. Different storage models: Document, key-value, column-family, and graph databases support different requirements.
5. Application-specific design: The data model can be selected according to the application's access patterns.

Not every NoSQL database has the same capabilities. Scalability, consistency, transactions, and performance depend on the particular product and its configuration.

4. DIFFERENCES BETWEEN RDBMS AND NOSQL

Feature	RDBMS	NoSQL
Data structure	Tables with rows and columns	Documents, key-value pairs, column families, or graphs
Schema	Usually predefined	Often flexible or model-dependent
Query language	Commonly SQL	Query methods vary by database
Relationships	Uses primary keys, foreign keys, and joins	Often uses embedded data, references, or graph relationships
Data consistency	Commonly supports strong transactional consistency	Consistency options vary by database
Transactions	Commonly supports ACID transactions	Transaction support varies by database
Scalability	Can scale vertically and, in some systems, horizontally	Many systems are designed for horizontal scaling

Data duplication	Normalization can reduce duplication	Some designs intentionally duplicate data for efficient access
Best suited for	Structured data and relational transactions	Use cases requiring flexible models or particular distributed-data capabilities
Examples	MySQL, PostgreSQL, Oracle Database	MongoDB, Redis, Cassandra, Neo4j

The main difference is that RDBMS focuses on relational tables and structured relationships, while NoSQL provides alternative data models for different application needs.

5. SCALABILITY

Scalability refers to a database's ability to handle increasing amounts of data, users, or requests while maintaining acceptable performance.

5.1 RDBMS Scalability

RDBMS systems can scale vertically by increasing the resources of a server, such as its memory, processing power, and storage.

Some relational databases also support horizontal scaling through techniques such as replication, partitioning, and distributed database architectures.

Example:

A paranormal investigation agency may initially manage a few hundred cases using MySQL. As the number of cases increases, the database can be optimized and its server resources expanded.

Advantages:

- Suitable for structured records and related information.
- Supports transactions and relational constraints.
- Provides powerful SQL queries for reporting and analysis.

Limitations:

- Complex joins and poorly optimized queries may become expensive as data grows.

- Scaling a relational database across multiple servers can require additional planning and configuration.

5.2 NoSQL Scalability

Many NoSQL databases are designed to distribute data and workloads across multiple servers. This approach is known as horizontal scaling.

Example:

If an agency collects large volumes of sensor readings, photographs' metadata, and activity logs from many investigation locations, a suitable NoSQL database may help manage the growing volume of distributed records.

Advantages:

- Many systems support horizontal scaling.
- Flexible data models can accommodate changing record structures.
- Some systems are designed for high-volume distributed workloads.

Limitations:

- Data distribution and partitioning require careful planning.
- Some queries involving complex relationships may be less straightforward than relational joins.
- Consistency and transaction capabilities vary by database.

Comparison: RDBMS is suitable for many structured business applications, while NoSQL may be preferable when its particular data model and distributed architecture better match the workload.

6. DATA CONSISTENCY

Data consistency means that information remains correct and follows the rules defined for the database.

6.1 RDBMS Consistency

RDBMS databases commonly support **ACID** transactions.

- **Atomicity:** A transaction completes entirely or is rolled back.
- **Consistency:** A transaction preserves the database's defined rules and constraints.
- **Isolation:** Concurrent transactions are controlled according to the database's isolation level.
- **Durability:** Committed changes remain stored even after a failure, subject to the system's durability guarantees.

Example:

When a client payment is recorded, the system can use a transaction to ensure that related payment updates are completed together. Constraints can also prevent a payment from referencing a nonexistent case.

This makes **RDBMS** suitable for financial records and applications where reliable transactions are important.

6.2 NoSQL Consistency

NoSQL consistency depends on the database system and its configuration.

Some NoSQL databases support strong consistency, while others offer configurable or eventual consistency for certain operations.

Eventual consistency means that replicas may temporarily contain different values but are expected to converge if updates stop and the system continues operating normally.

Example:

An agency's distributed monitoring system may store activity logs across several locations. If the selected database supports eventual consistency, a new log entry may take some time to appear on every replica.

This may be acceptable for certain monitoring records, but it may be unsuitable for information that requires immediate, reliable agreement.

Comparison: RDBMS is often a natural choice for applications requiring relational integrity and transactions. NoSQL can also provide strong consistency and transactions in some systems, so the choice should be based on the actual product and requirements.

7. PERFORMANCE

Database performance refers to how quickly and efficiently a database processes operations such as inserting, updating, searching, and retrieving data.

7.1 RDBMS Performance

RDBMS can perform efficiently when tables, indexes, and SQL queries are designed properly.

Advantages:

- Efficient querying of structured data.
- Powerful joins and aggregate functions.
- Indexes help retrieve records quickly.
- Suitable for transactional operations and complex reports.

Example:

The agency can use SQL joins to generate a report containing a case ID, client name, assigned investigators, payment status, and final verdict.

However, performance may decrease if queries involve large tables, unnecessary joins, or missing indexes.

7.2 NoSQL Performance

NoSQL databases can perform efficiently when their data model matches the application's access patterns.

Advantages:

- Some document databases can retrieve complete documents without several joins.
- Key-value databases can provide fast access to values using known keys.
- Distributed databases can handle large volumes of requests across multiple machines.

Example:

A document database may store an investigation report together with its summary and related metadata. Retrieving the complete document may be convenient when the application usually needs all those details together.

However, performance depends on indexing, data design, hardware, query patterns, and workload. NoSQL is not automatically faster than **RDBMS**.

8. USE CASES OF RDBMS AND NOSQL

8.1 Use Cases of RDBMS

RDBMS is commonly used in applications that require structured records, relationships, and reliable transactions.

Examples include:

1. **Banking systems:** Managing accounts, transactions, and balances.
2. **College management systems:** Managing students, courses, attendance, and marks.
3. **E-commerce systems:** Managing customers, orders, products, and payments.
4. **Hospital management systems:** Managing patient records, appointments, and billing.
5. **Agency management systems:** Managing clients, investigations, reports, and payments.

8.2 Use Cases of NoSQL

NoSQL is useful when a particular non-relational data model fits the application's requirements.

Examples include:

1. **Real-time applications:** Using suitable key-value databases for frequently accessed data.
2. **Content management:** Storing flexible documents with different fields.
3. **Large distributed workloads:** Managing high volumes of data across multiple servers.
4. **Social networks:** Representing relationships using graph databases where graph queries are useful.
5. **Sensor and activity logs:** Storing large amounts of distributed event data using a suitable database.

The best choice depends on the data structure, consistency requirements, access patterns, and expected workload.

9. WHY ORGANIZATIONS CHOOSE DIFFERENT DATABASE TECHNOLOGIES

Organizations choose database technologies according to their business requirements rather than using the same database for every situation.

9.1 Data Structure

If the data is highly structured and contains clear relationships, an RDBMS may be a suitable choice.

If the data has a flexible structure or fits a document, key-value, column-family, or graph model, a NoSQL database may be more appropriate.

9.2 Consistency Requirements

Financial transactions and other critical records often require strong transactional guarantees and carefully enforced relationships.

Applications that can tolerate temporary differences between distributed replicas may choose a database that offers eventual consistency, provided that this trade-off is acceptable.

9.3 Scalability Requirements

An organization expecting rapid growth in distributed data may select a database designed for horizontal scaling.

Other organizations may prefer an RDBMS because its relational features and transaction support better match their workload.

9.4 Performance Requirements

A database should be selected according to the operations the application performs most frequently.

For example, a system that requires complex reporting and joins may benefit from an RDBMS, while a system that frequently retrieves complete documents may benefit from a document database.

9.5 Cost and Maintenance

Organizations also consider licensing, infrastructure, staff expertise, backup, security, monitoring, and maintenance costs.

The most suitable database is the one that meets the application's requirements while remaining practical to operate.

10. REAL-WORLD EXAMPLES

The following are examples of database technologies and their typical applications.

Organization or Application	Database Technology / Example	Why It May Be Used
Banking application	MySQL or PostgreSQL	Relational records, transactions, and structured reporting
College management system	Microsoft SQL Server	Related student, course, attendance, and marks records
Content platform	MongoDB	Flexible document structures
Real-time caching service	Redis	Fast key-based access to frequently used data
Large distributed data service	Apache Cassandra	Distributed data storage and high-volume workloads
Relationship analysis application	Neo4j	Queries involving connected entities and relationships

These examples illustrate typical uses rather than claiming that every organization uses only the listed database. Large organizations may use several database technologies together.

11. SUITABILITY FOR THE PARANORMAL INVESTIGATION AGENCY MANAGEMENT SYSTEM

The proposed Paranormal Investigation Agency Management System contains 11 related tables: Clients, Investigators, Locations, Case_Categories, Equipment, Cases, Case_Investigator, Case_Equipment, Evidence, Reports, and Payments.

Most of its information is structured and connected through primary keys and foreign keys.

For example:

- **One client can report multiple cases.**
- **A case can involve several investigators.**
- **A case can use multiple equipment items.**
- **A case can generate multiple evidence records.**
- **Reports and payments are linked to cases.**

These relationships make an RDBMS a suitable choice for the current project.

Why RDBMS Is Suitable

1. Structured data

The system has clearly defined attributes, such as client_id, case_id, investigation_id, payment_status, and case_status.

2. Relationships between tables

Primary keys and foreign keys can connect cases with clients, locations, investigators, evidence, and payments.

3. Data integrity

Constraints can help prevent invalid case references, missing required information, and invalid payment amounts.

4. Transaction support

Transactions can help maintain consistency when recording payments or updating related case information.

5. Reporting and analysis

SQL joins, aggregate functions, and filters can generate reports on case status, investigator assignments, equipment usage, and payments.

When NoSQL Could Be Useful

If the agency later collects large amounts of flexible evidence metadata, device logs, or differently structured investigation documents, a NoSQL database may be useful for those specific requirements.

For example, MongoDB could store documents containing varying metadata about audio recordings, photographs, and sensor readings.

However, adopting NoSQL would depend on the application's scale and requirements. It is not necessary simply because the system stores evidence.

Final Recommendation

For the current project, a relational database such as MySQL or PostgreSQL is recommended because the system has structured information, clearly defined relationships, and a need for reliable records and reporting.

A NoSQL database could be considered later if the agency develops requirements that are better served by a particular NoSQL data model.

12. CONCLUSION

RDBMS and **NoSQL** databases are both important technologies for modern applications. RDBMS organizes information into related tables and provides relational constraints, SQL querying, and transactional capabilities. NoSQL databases support different data models and may offer flexible schemas and distributed scaling suited to particular workloads.

The choice between them depends on data structure, consistency, scalability, performance, and business requirements. Neither approach is universally better in every situation.

For the Paranormal Investigation Agency Management System, RDBMS is the more suitable starting point because the project contains structured data and relationships between clients, investigators, cases, equipment, evidence, reports, and payments.

A NoSQL database may be useful for additional requirements involving flexible evidence metadata or large volumes of distributed activity records. The database should ultimately be selected according to the actual needs of the application.